

emory Forensics & Volatile Data Analysis

Volatile memory contains critical forensic artifacts such as live processes, injected code regions, network connections, and credential remnants that may not exist on disk. Traditional static analysis is often insufficient for detecting in memory threats, especially in malware and incident response scenarios.

This project aims to provide a practical memory forensics workflow that is accessible, modular, and suitable for lab or educational environments. The core goals were

- Support multi-OS dump analysis (Windows/Linux/macOS)
- Automate major forensic checks (processes, injection, network, secrets, YARA)
- Provide both CLI and GUI interfaces
- Produce structured evidence reports for documentation

Project Objectives

The project was designed to meet the following objectives :

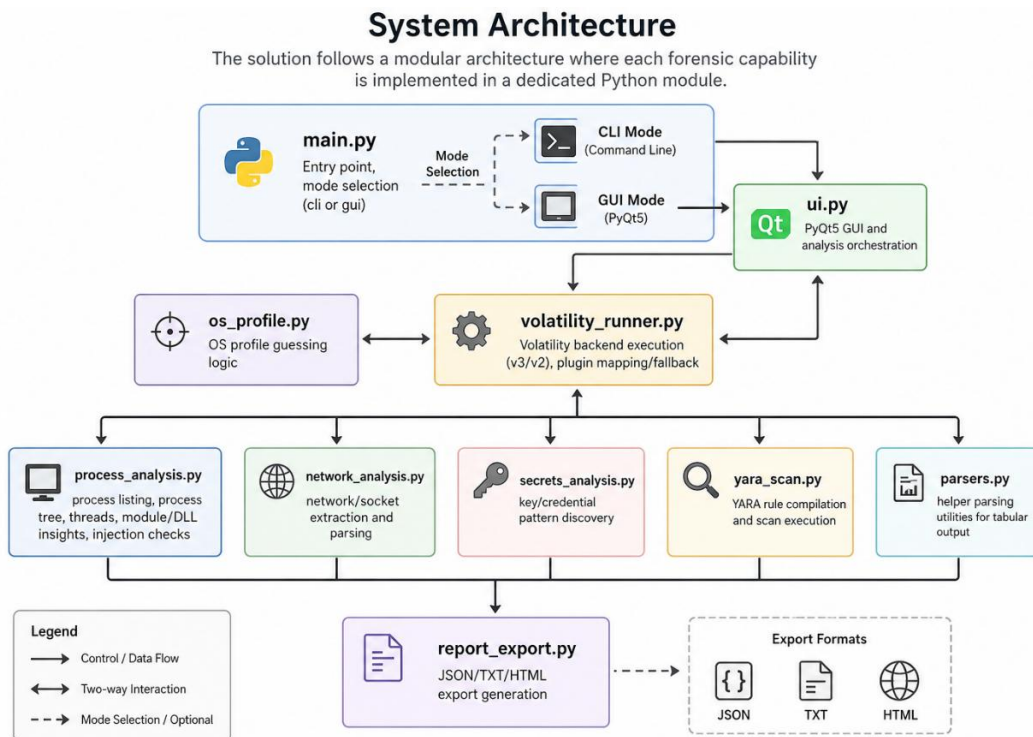
- Parse and analyze memory dumps using Volatility plugins
- Detect suspicious in-memory behavior (e.g., potential code injection)
- Recover network artifacts and socket details
- Identify possible secrets and credential indicators
- Apply YARA signatures for known threat matching
- Export analysis findings into shareable report formats
- Improve analyst workflow through a clean and readable GUI

System Architecture

The solution follows a modular architecture where each forensic capability is implemented in a dedicated Python module

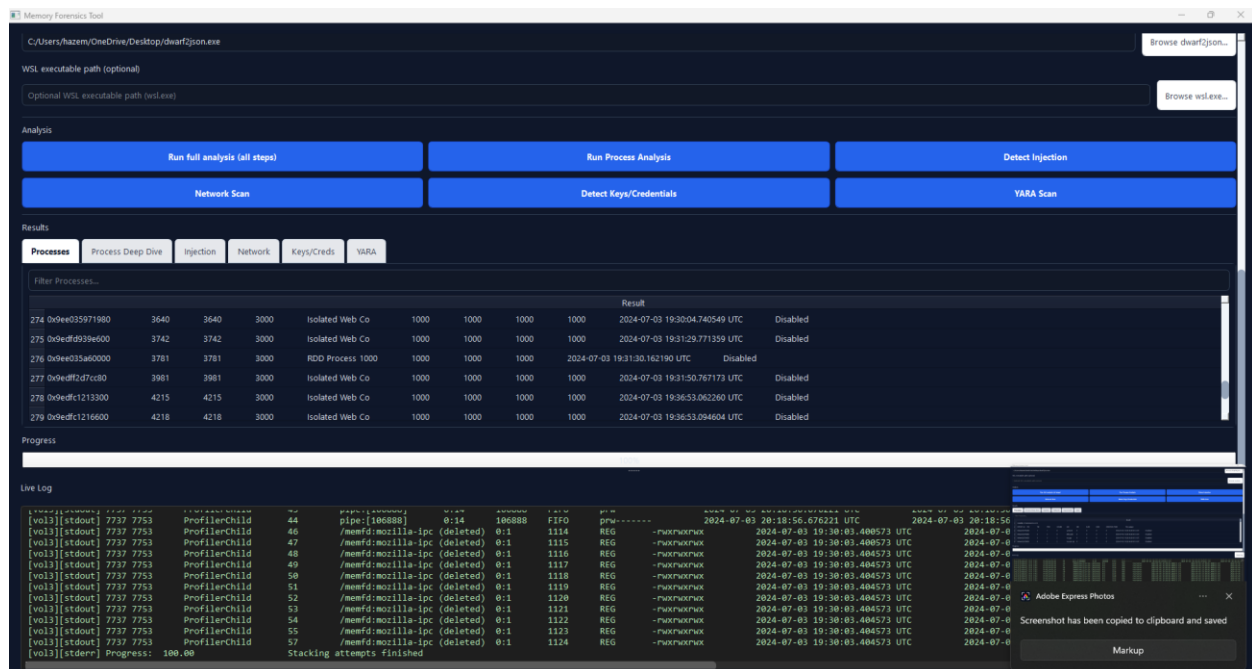
- **main.py:** Entry point, mode selection (cli or gui)
Main entry point. Lets user choose cli or gui mode, then starts analysis flow.
- **ui.py:** PyQt5 GUI and analysis orchestration
PyQt5 graphical interface. Handles loading dumps, running analysis tasks, showing results in tabs, and exporting reports.
- **volatility_runner.py:** Volatility backend execution (v3/v2), plugin mapping/fallback
Core execution layer for Volatility. Manages Volatility 3/2 config, plugin mapping, fallback execution, and command running.
- **os_profile.py:** OS profile guessing logic
Detects likely target OS (windows/linux/mac) from memory artifacts using lightweight plugin probes.
- **process_analysis.py:** process listing, process tree, threads, module/DLL insights, injection checks
Process-focused analysis: process list, process tree, thread/module records, and injection/malfind-based suspicious findings.
- **network_analysis.py:** network/socket extraction and parsing
Network-focused analysis: extracts and parses socket/connection data from memory plugins.
- **secrets_analysis.py:** key/credential pattern discovery
Searches memory-derived data for potential secrets/credentials (keys, tokens, passwords, etc.).
- **yara_scan.py:** YARA rule compilation and scan execution
Shared parsing helpers to convert Volatility text/table output into structured Python records
Compiles YARA rules and scans memory dumps, then formats match results.

- **report_export.py:** JSON/TXT/HTML export generation
Builds final forensic report object and exports to JSON, TXT, and HTML formats
- **parsers.py:** helper parsing utilities for tabular output
Shared parsing helpers to convert Volatility text/table output into structured Python records.
- **forensics_logging.py**
Logging setup and helper functions for runtime errors and forensic/Volatility activity tracking.
- **rules.yar**
YARA signatures used by yara_scan.py to detect known suspicious patterns.



Methodology and Workflow

1. User loads memory dump file (CLI path input or GUI file dialog).
2. Tool performs best-effort OS profiling and allows manual override.
3. Selected modules run individually or through full analysis mode:
 - Process analysis
 - Injection detection
 - Network scan
 - Secrets detection
 - YARA scan
4. Findings are shown in dedicated GUI tabs (or CLI output).
5. User exports final report to JSON/TXT/HTML.



Key Features and Capabilities

- **Multi-OS Support:** Windows, Linux, and macOS-oriented plugin paths
- **Backend Flexibility:** Volatility 3 plus Volatility 2 compatibility mode
- **Process Forensics:** process lists, trees, thread/module context
- **Injection Indicators:** malfind-based suspicious region summaries
- **Network Artifacts:** socket/connection extraction and record parsing
- **Secret Detection:** likely keys/tokens/credentials from memory content
- **YARA Matching:** rule-based malware indicator checks
- **Structured Reporting:** reproducible report generation in multiple formats